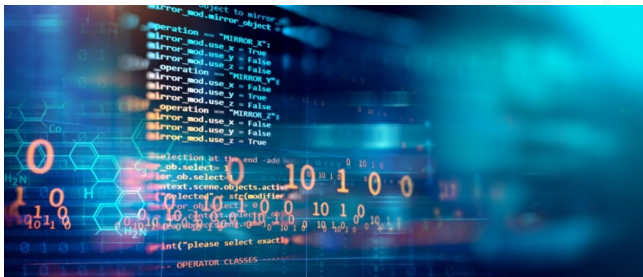


Programación Funcional en Haskell

Tecnología de Programación



Adolfo Muñoz – Daniel Martín – Juan Magallón
Grado en Ingeniería Informática



**Universidad
Zaragoza**



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

Funciones en Haskell

A continuación verás una serie de programas en el lenguaje Haskell.

Intenta averiguar qué hacen las distintas funciones que aparecen en ellos, y qué conceptos y técnicas de programación funcional se han utilizado para implementarlas.

En algunos casos aparecen distintas implementaciones.
Compara las diferencias entre ellas.

```
1 f :: Int -> Int
```

```
1 f x = if x>=0  
2     then x  
3     else -x
```

```
1  -- f :: Int -> Int
```

```
1  f x = if x>=0
```

```
2      then x
```

```
3      else -x
```

```
1  f :: Int -> Int
2
3  f  x = if x==0
4         then 1
5         else x * f (x-1)
```

```
1 f :: Int -> Int
2
3 f x = if x==0
4       then 1
5       else x * f (x-1)
```

```
1 f' 0 = 1
2 f' x = x * f' (x-1)
```

```
1  f :: Double -> Int -> Double
2
3  f  b n = if n==0
4           then 1
5           else b * f b (n-1)
```

```
1 f :: Double -> Int -> Double
```

```
2
```

```
3 f    b n = if n==0  
4         then 1  
5         else b * f b (n-1)
```

```
1 f'    b 0 = 1
```

```
2 f'    b n = b * f' b (n-1)
```

```
1 f :: Double -> Int -> Double
```

```
2
```

```
3 f    b n = if n==0  
4         then 1  
5         else b * f b (n-1)
```

```
1 f'   b 0 = 1
```

```
2 f'   b n = b * f' b (n-1)
```

```
1 f''  _ 0 = 1
```

```
2 f''  b n = b * f'' b (n-1)
```

```
1  f :: Int -> Double -> Double
2
3  f  n b = if n==0
4           then 1
5           else b * f (n-1) b
```

```
1 f :: Int -> Double -> Double
```

```
2
```

```
3 f    n b = if n==0  
4         then 1  
5         else b * f (n-1) b
```

```
1 f'  0 _ = 1
```

```
2 f'  n b = b * f' (n-1) b
```

```
1 f :: Int -> Double -> Double
```

```
2
```

```
3 f    n b = if n==0  
4         then 1  
5         else b * f (n-1) b
```

```
1 f'  0 _ = 1
```

```
2 f'  n b = b * f' (n-1) b
```

```
1 sqr b = f 2 b
```

```
2
```

```
3 sqr    = f 2
```

```
1  aux k n = if k==n
2             then [k]
3             else if mod n k == 0
4                     then k:(aux (k+1) n)
5                     else (aux (k+1) n)
6
7  f n = aux 1 n
```

```
1  aux n c = chr (ord c + n)
2
3  f  n s      = if s==" "
4                then " "
5                else (aux n (head s)):(f n (tail s))
```

```
1  aux n c = chr (ord c + n)
2
3  f  n s      = if s=="
4                  then ""
5                  else (aux n (head s)):(f n (tail s))
6
7  f'  _  ""    = ""
8  f'  n s      = (aux n (head s)):(f n (tail s))
```

```
1 aux n c = chr (ord c + n)
```

```
2
```

```
3 f    n s      = if s==" "  
4               then ""  
5               else (aux n (head s)):(f n (tail s))
```

```
1 f' _ ""      = ""
```

```
2 f' n s      = (aux n (head s)):(f n (tail s))
```

```
1 f'' _ ""     = ""
```

```
2 f'' n (x:xs) = (aux n x):(f n xs)
```

```
1  f  x ys      = if null ys
2                  then [x]
3                  else if x < head ys
4                      then x:ys
5                      else (head ys):(f x (tail ys))
```

```
1  f  x ys      = if null ys
2                  then [x]
3                  else if x < head ys
4                      then x:ys
5                      else (head ys):(f x (tail ys))
```

```
1  f'  x []      = [x]
2  f'  x ys      = if x < head ys
3                  then x:ys
4                  else (head ys):(f' x (tail ys))
```

```
1  f  x ys      = if null ys
2                  then [x]
3                  else if x < head ys
4                      then x:ys
5                      else (head ys):(f x (tail ys))
```

```
1  f'' x []      = [x]
2  f'' x (y:ys)  = if x < y
3                  then x:y:ys
4                  else y:(f'' x ys)
```

```
1  f  x ys      = if null ys
2                  then [x]
3                  else if x < head ys
4                      then x:ys
5                      else (head ys):(f x (tail ys))
```

```
1  f'' x []      = [x]
2  f'' x l@(y:ys) = if x < y
3                  then x:l
4                  else y:(f'' x ys)
```

```
1  aux mm xs = if null xs
2    then mm
3    else aux (min (fst mm) (head xs), max (snd mm) (head xs))
4              (tail xs)
5
6  f xs      = if null xs
7              then error "lista vacia"
8              else aux (head xs, head xs) (tail xs)
```

```
1  aux mm xs = if null xs
2    then mm
3    else aux (min (fst mm) (head xs), max (snd mm) (head xs))
4              (tail xs)
5
6  f  xs      = if null xs
7              then error "lista vacia"
8              else aux (head xs, head xs) (tail xs)
9
10 f' []      = error "lista vacia"
11 f' xs      = aux (head xs, head xs) (tail xs)
```

```
1  aux mm xs = if null xs
2    then mm
3    else aux (min (fst mm) (head xs), max (snd mm) (head xs))
4              (tail xs)
5
6  f xs      = if null xs
7              then error "lista vacia"
8              else aux (head xs, head xs) (tail xs)
9
10 f' []      = error "lista vacia"
11 f' (x:xs)  = aux (x,x) xs
```

```
1 aux mm [] = mm
2 aux (mn,mx) (x:xs) = aux (min mn x,max mx x) xs
```

```
1 f' [] = error "lista vacia"
2 f' (x:xs) = aux (x,x) xs
```

```
1  f :: [a] -> Int
2
3  f []      = 0
4  f (_:xs) = 1 + f xs
```

```
1 f :: [a] -> Int
```

```
2
```

```
3 f []      = 0
```

```
4 f (_:xs) = 1 + f xs
```

```
1 f xs      = sum (map (\x -> 1) xs)
```

```
1 f :: [a] -> Int
```

```
2
```

```
3 f []      = 0
```

```
4 f (_:xs) = 1 + f xs
```

```
1 f xs      = sum (map (\x -> 1) xs)
```

```
1 f xs      = sum ( (map (\x -> 1)) xs )
```

```
1 f :: [a] -> Int
```

```
2
```

```
3 f []      = 0
```

```
4 f (_:xs) = 1 + f xs
```

```
1 f xs      = sum (map (\x -> 1) xs)
```

```
1 f xs      = sum ( (map (\x -> 1)) xs )
```

```
1 f xs      = (sum . (map (\x -> 1))) xs
```

```
1 f :: [a] -> Int
```

```
2
```

```
3 f []      = 0
```

```
4 f (_:xs) = 1 + f xs
```

```
1 f xs      = sum (map (\x -> 1) xs)
```

```
1 f xs      = sum ( (map (\x -> 1)) xs )
```

```
1 f xs      = (sum . (map (\x -> 1))) xs
```

```
1 f xs      = (sum . map (\x -> 1)) xs
```

```
1 f :: [a] -> Int
```

```
2
```

```
3 f []      = 0
```

```
4 f (_:xs) = 1 + f xs
```

```
1 f xs      = sum (map (\x -> 1) xs)
```

```
1 f xs      = sum ( (map (\x -> 1)) xs )
```

```
1 f xs      = (sum . (map (\x -> 1))) xs
```

```
1 f xs      = (sum . map (\x -> 1)) xs
```

```
1 f          = sum . map (\x -> 1)
```

```
1  f :: [a] -> [a]
2
3  f []      = []
4  f (x:xs) = f xs ++ [x]
```

```
1  f :: [a] -> Int -> a
2
3  f (x:_) 1 = x
4  f (_:xs) i = f xs (i-1)
5  f _ _ = error "index out of bounds"
```

```
1 f :: (Eq a) => [a] -> Bool
2
3 f [] = True
4 f [_] = True
5 f xs = (head xs) == (last xs) && (f (init (tail xs)))
```

```
1  f :: (Eq a) => [a] -> Bool
2
3  f [] = True
4  f [_] = True
5  f xs = (head xs) == (last xs) && (f $ init $ tail xs)
```

```
1  f :: [a] -> a
2
3  f [x]      = x
4  f (_:xs) = f xs
5
6  f'      = head . reverse
```

```
1  f :: [a] -> a
2
3  f [x, _] = x
4  f (_:xs) = f xs
5
6  f' (x:(_:[])) = x
7  f' (_:xs)     = f' xs
8
9  f'' x = reverse x !! 1
10
11 f''' = last . init
12
13 f'''' = head . tail . reverse
```

```
1 f :: a -> [a]
2
3 f x = (x:(f x))
```

```
1 f :: (a -> a) -> a -> [a]
2
3 f g x = (x:(f g (g x)))
```

```
1  f :: (a -> b) -> [a] -> [b]
2
3  f _ []      = []
4  f t (x:xs) = (t x):(f t xs)
```

```
1  f :: (a -> Bool) -> [a] -> [a]
2
3  f _ [] = []
4  f p (x:xs) = if p x
5                then x:(f p xs)
6                else  (f p xs)
```

```
1 f :: (a -> Bool) -> [a] -> [a]
```

```
2
```

```
3 f _ [] = []
```

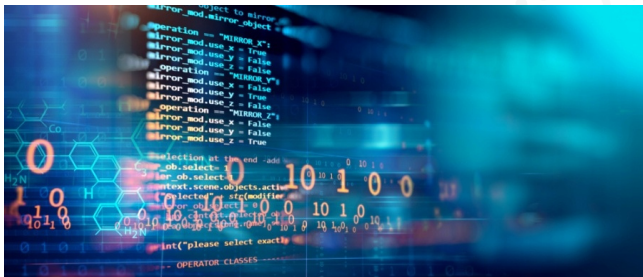
```
4 f p (x:xs) = if p x
5               then x:(f p xs)
6               else  (f p xs)
```

```
1 f _ [] = []
```

```
2 f p (x:xs) = if p x
3               then x:r
4               else  r
5               where
6                 r = f p xs
```

Programación Funcional en Haskell

Tecnología de Programación



Adolfo Muñoz – Daniel Martín – Juan Magallón
Grado en Ingeniería Informática



**Universidad
Zaragoza**



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza